

DevOps Capstone Project – Retail E-Commerce Order Management Platform

Project Scenario

You are part of a DevOps engineering team working for a:

Retail E-Commerce Platform

The engineering team is building a small internal application that exposes APIs for:

Order Service Health Check
Order Status API
Inventory Status API

The organization wants:

Agile planning
Source control management
CI automation
Automated validation

using:

Trello
GitHub
GitHub Actions
Python Flask



Agile Planning Requirement

Using Trello, break down the following Epic into:

Features
User Stories
Tasks



Build and automate CI workflow for Retail Order Management Platform

Trello Board Structure

Create lists such as:

Backlog
Features
User Stories
Tasks
In Progress
Completed

Expected Planning Areas

The planning should realistically cover:

API development
Order processing
Inventory validation
Application monitoring
Testing
Repository management
CI pipeline setup
Build verification

The breakdown should reflect how a real retail engineering team would execute the project.

Application Code

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Retail Order Management Platform"

@app.route("/health")
def health():
    return "Order Service Healthy"

@app.route("/order-status")
def order_status():
    return "Order Processing Successfully"

@app.route("/inventory")
def inventory():
    return "Inventory Available"

if __name__ == "__main__":
    app.run()
```

requirements.txt

```
flask
pytest
```

test_app.py

```
from app import app

def test_home():
    client = app.test_client()
    response = client.get("/")
    assert response.status_code == 200

def test_health():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200

def test_order_status():
    client = app.test_client()
    response = client.get("/order-status")
    assert response.status_code == 200

def test_inventory():
    client = app.test_client()
    response = client.get("/inventory")
    assert response.status_code == 200
```



Source Control Requirement

Push the application into a GitHub repository using proper repository structure and commit practices.



CI Pipeline Requirement

Create a GitHub Actions workflow:

```
.github/workflows/python-app.yml
```

The workflow should:

```
Install dependencies
Run automated tests
Validate build success
```



GitHub Actions Workflow

```
name: Retail Order Management CI Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  build-and-test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout Code
```

```
        uses: actions/checkout@v4
```

```
      - name: Set Up Python
```

```
        uses: actions/setup-python@v5
```

```
        with:
```

```
          python-version: '3.10'
```

```
      - name: Install Dependencies
```

```
        run: |
```

```
          pip install -r requirements.txt
```

```
      - name: Run Tests
```

```
        run: |
```

```
          pytest
```



Verification

Validate the following:

Code successfully pushed to GitHub
GitHub Actions workflow triggered automatically
Pipeline execution completed successfully
All tests passed

Deliverables

Deliverable
Trello board screenshot
Epic / Feature / Story / Task breakdown
GitHub repository screenshot
Successful GitHub Actions pipeline screenshot
Flask application source code
